

L'esercizio di oggi verte sui meccanismi di pianificazione dell'utilizzo della CPU (o processore). In ottica di ottimizzazione della gestione dei processi, abbiamo visto come lo scheduler si sia evoluto nel tempo per passare da approccio mono-tasking ad approcci multi-tasking.

Traccia: Si considerino 4 processi (P1, P2, P3, P4) con i tempi di esecuzione e di attesa input/output dati in tabella. I processi arrivano alla CPU in ordine P1, P2, P3, P4. Individuare il modo più efficace per la gestione e l'esecuzione dei processi, tra i metodi già visti a lezione. Abbozzare un diagramma che abbia sulle ascisse il tempo passato da un istante «0» e sulle ordinate il nome del Processo.

Tabla de datos del problema:

Processo	Tempo di esecuzione	Tempo di attesa	Tempo di esecuzione dopo attesa
P1	3 secondi	1 secondo	1 secondo
P2	1 secondo	2 secondi	-
P3	2 secondi	-	-
P4	4 secondi	1 secondo	-

Llegan a la CPU en el siguiente orden: P1 → P2 → P3 → P4

Ates de continuar con el ejercicio prendo nota de lo siguiente:

A. Mono-tasking (Un solo proceso)

En un sistema **Mono-tasking**, solo puede existir **un proceso ejecutándose**.

Imagina esta situación:

P1 usa la CPU durante 3 segundos.

↓

Ahora necesita leer datos del disco (I/O).

↓

Espera 1 segundo

Que hace la CPU en ese minuto?: No hace nada
VENTAJAS!

1. Muy simple
2. Fácil de implementar.

DESVENTAJAS.

1. La CPU pasa mucho tiempo sin trabajar.

2. Se desaprovecha el hardware.

B. Multi-tasking (Varios procesos)

Ahora el sistema operativo piensa de esta manera:

Si P1 esta esperando al disco.... Por que mejor no ejecuto otro programa?

La gran idea del **Multi-tasking** es:

Mientras un proceso espera, otro aprovecha la CPU.

Por eso el rendimiento mejora mucho.

C. Time-sharing (Evolución del Multi-tasking)

Aqui esta la diferencia importante.

En el Multitasking básico. Normalmente un proceso sigue ejecutándose hasta que:

- Termina
- O necesita hacer I/O.

Por ejemplo imagina que:

P1 Necesita 50 segundos.

P2 Necesita 1 segundo...

P1.....

P2 espera.

Esto que quiere decir?

Quiere decir que Aunque exista Multi-Tasking, P2 Puede esperar mucho.

Entonces es aqui donde nace el **Time-sharing**.

¿Qué hace el Time-sharing?

El sistema operativo divide el tiempo en pequeños trozos llamados **Quantum**.

Por ejemplo: quantum = 2 segundos

Da la sensación de que **todos los procesos se ejecutan al mismo tiempo**, aunque en realidad la CPU cambia tan rápido entre ellos que el usuario lo percibe como simultáneo.

Diferencia entre Multi-tasking y Time-sharing

Muchos estudiantes creen que son lo mismo, pero no exactamente.

Multi-tasking

Permite varios procesos.

Time-sharing

Es una evolución del Multi-tasking.

Multi-tasking

Cambia de proceso cuando uno termina o espera I/O.

Optimiza el uso de la CPU.

Time-sharing

Cambia de proceso cada cierto tiempo (Quantum).

Optimiza el uso de la CPU y mejora la respuesta al usuario.

Por esto podemos decir que **Todo sistema de Time-sharing son Multi-tasking, pero no todo sistema Multi-tasking utiliza Time-sharing.**

Dicho esto puedo seguir con el ejersicio W4D1.

¿Cuál es el método más eficaz?.

El metodo mas eficaz seria el Time-sharing, porque es la evolución del Multi-Taskin.

Por que?

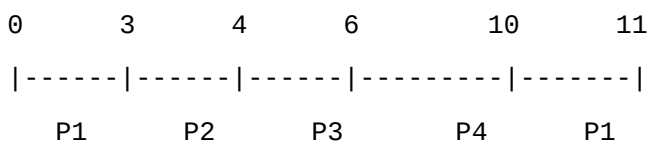
Porque:

- permite ejecutar varios procesos;
- aprovecha los tiempos de espera de I/O;
- evita que la CPU permanezca inactiva;
- reparte el tiempo del procesador entre los procesos, mejorando la respuesta del sistema.

Diagrama temporal (esquema)

Como nos han dado un **quantum**, no hace falta dibujar un Round Robin detallado. Lo importante es mostrar que cuando un proceso espera I/O, otro aprovecha la CPU.

Tempo



Donde:

- **0-3:** P1 usa la CPU.
- **3-4:** P1 está en espera de I/O; la CPU ejecuta P2.
- **4-6:** la CPU ejecuta P3.
- **6-10:** la CPU ejecuta P4.
- **10-11:** P1, que ya terminó el I/O, vuelve a la CPU para completar su último segundo.